

Can you please let me know what is incorrect in below DFS code. It's giving correct result AFAIK, but I don't know when it will fail.

10

```
graph1 = {
    'A': ['B','S'],
    'B': ['A'],
    'C': ['B','F','S'],
    'D': ['C'],
    'E': ['C','W'],
    'F': ['C','D'],
    'G': ['F','S'],
    'H': ['E','D'],
    'S': ['A','E','W']
}

visited = []

def dfs(graph,node):
    global visited
    if node not in visited:
        visited.append(node)
        for n in graph[node]:
            dfs(graph,n)

dfs(graph1,'A')
print(visited)
```

Output:

['A', 'B', 'S', 'C', 'D', 'E', 'W', 'G', 'F']

python depth-first-search Edit tags

Share Edit Follow Close Flag Protect

edited May 9, 2017 at 16:13

asked Apr 15, 2017 at 19:28

Arun Leni

7,188

6

58

90

Vicky

161

1

1

6

2

First of all, don't use `global`. I avoid them as much as possible! – [wileM_Van Orsen](#) Apr 15, 2017 at 19:24

2

DFS is a search algorithm, but there is no target defined you are looking for. – [wileM_Van Orsen](#) Apr 15, 2017 at 19:28

Thanks for the response.. `visited = []` def dfs(graph,node):visited: if node not in visited: visited.append(node) for n in graph[node]: dfs(graph,n,visited)

I'm also not convinced that the output you show is generated by the data you show - I don't see where S, H and G come from? (But I could possibly be wrong here)

– [DavidW](#) Apr 15, 2017 at 19:32

Hi Wilem, if you checkout the link [eddiemem.com/post/...](#), there a sample code for the DFS is given but if I change the graph as given in youtube tutorial ["youtube.com/eddiemem7x"](#) I am not getting the result as mentioned. So I thought of writing my own version based on the tutorial. – [Vicky](#) Apr 15, 2017 at 19:34

11 Answers

Sorted by
Relevance

Date modified (newest first)

very basic code ig

1

```
def dfs(src, vertex):
    dfs[]
    stack=[]

    stack.append(vertex)
    while len(stack)!=0:
        top=stack.pop()
        if top not in dfs:
            dfs.append(top)

            for i in dfs[top]:
                if i not in visited:
                    stack.append(i)
                    stack.append(i)
                    break

    return dfs

print(dfs(src,'A'))
```

Share Edit Follow Delete Flag

edited Nov 25, 2023 at 22:34

answered Nov 25, 2023 at 19:43

Shafayett Karim

1,047

0

1

2

Without recursion:

13

```
def dfs(graph, node):
    visited = set([node,])
    output = []
    stack = [node]
    while stack:
        node = stack[-1]
        if node not in visited:
            output.append(node)
            visited.add(node)
            remove_from_stack = True
            for next_node in graph[node]:
                if next_node not in visited:
                    stack.append(next_node)
                    remove_from_stack = False
                    break
            if remove_from_stack:
                stack.pop()
            return output

    print (dfs(graphs, 'A'))
```

Output:

['A', 'B', 'S', 'C', 'D', 'E', 'W', 'G', 'F']

Share Edit Follow Flag

edited Aug 16, 2022 at 14:33

answered Aug 15, 2018 at 20:24

Gulzar

76,26

30

133

229

gastano

897

1

16

32

1

Why extend instead of append ? 2. Why is visited a list and not a set? – [Gulzar](#) Aug 4, 2022 at 13:09

append is fine. I prefer to have visited as it is so it can keep track of the visit order – [gastano](#) Aug 16, 2022 at 10:55

1

If it is a list, you lose `o(n)` in performance – [Gulzar](#) Aug 16, 2022 at 13:02

Also note that next is a reserved word in Python – [Gulzar](#) Aug 16, 2022 at 13:02

agreed. I added a variable output to keep the right visited order – [gastano](#) Aug 16, 2022 at 13:47

Here's an iterative (non-recursive) implementation of a DFS:

5

```
def dfs_iterative(graph, start_vertex):
    visited = set()
    traversal = []
    stack = [start_vertex]
    while len(stack) > 0:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            traversal.append(vertex)
            stack.extend(reversed(graph[vertex])) # add vertex in the same order as visited

    return traversal

test_graph = {
    'A': ['B','S'],
    'B': ['A'],
    'C': ['B','F','S'],
    'D': ['C'],
    'E': ['C','W'],
    'F': ['C','D'],
    'G': ['F','S'],
    'H': ['E','D'],
    'S': ['A','E','W']
}

print(dfs_iterative(test_graph, 'A'))
```

Output:

['A', 'B', 'S', 'C', 'D', 'E', 'W', 'G', 'F']

Share Edit Follow Flag

edited Aug 4, 2022 at 13:20

answered Mar 15, 2020 at 22:38

Gulzar

76,26

30

133

229

Saurabh

6,306

5

41

54

A recursive implementation:

0

```
def dfs(u, v, visited=[]):
    """Recursion version for depth-first search (DFS).
    Args:
        G: a graph
        u: start
        visited: a list containing all visited nodes in G
    Returns:
        ... visited
    visited.append(u)
    for v in G[u]:
        if v not in visited:
            dfs(G, v, visited)
    return visited
```

An iterative implementation using a stack:

```
def dfs_iterative(G, u, visited=[]):
    stack = [u, 1, Initialize]
    while stack:
        u = stack.pop()
        if u not in visited:
            visited.append(u)
            stack.extend(reversed(G[u])) # reversed() is used as the same traversal as recursive DFS
    return visited
```

Share Edit Follow Flag

edited Jul 8, 2021 at 7:01

answered Jul 6, 2021 at 9:51

user

1,842

1

12

31

```
def DFS(graph, pointer, visit):
    visit.add(pointer)
    print(pointer, end=" ")

    for travel in graph[pointer]:
        if travel not in visit:
            DFS(graph, travel, visit)

graph = {
    'A': set(['1', '2', '3']),
    '1': set(['4', '2']),
    '2': set(['3', '4', '5']),
    '3': set(['4']),
    '4': set(['5'])
}

print("DFS for graph: ", graph)
DFS(graph, 'A', visit)
```

0

graph:

DFS for graph: { '0': { '1', '3', '2' }, '1': { '0', '2' }, '2': { '1', '0', '4' }, '3': { '0' }, '4': { '1' }, '5': { '2' } }

0 1 2 3 4 5

Share Edit Follow Flag

edited Mar 23, 2021 at 6:23

answered Mar 23, 2021 at 6:13

Amisha Sahu

89

0

6

Here is a more versatile algorithm, the one asked in the question works only for undirected graphs. But this hopefully works for both them.

0

Check it out

```
graphs = {
    'A': ['B','S'],
    'B': ['A'],
    'C': ['B','S'],
    'D': ['C'],
    'E': ['C','W'],
    'F': ['C','D'],
    'G': ['F','S'],
    'H': ['E','D'],
    'S': ['A','E','W']
}

visited = []

def dfs_visit(graph, v):
    global visited
    for n in graph[v]:
        if n not in visited:
            visited.append(v)
            dfs_visit(graph, n)

def dfs(graph):
    global visited
    for v in graph:
        if v not in visited:
            visited.append(v)
            dfs_visit(graph, v)

dfs(graphs)
print(visited)
```

Share Edit Follow Flag

answered Jul 2, 2020 at 13:51

Ramesh2

1

1

3

DFS implementation in Python

0

```
from collections import defaultdict

class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v)

        for i in self.graph[v]:
            if visited[i] == False:
                self.DFSUtil(i, visited)

    def DFS(self):
        v = len(self.graph)
```


Welcome back! If you found this question useful, don't forget to vote both the question and the answers up. [close this message](#)

```
# Driver code
# Create a graph given in the above diagram
g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 0)

print("Following is Depth First Traversal")
g.DFS()
```

Source : [this](#)

Share Edit Follow Flag

edited Feb 4, 2019 at 11:44
Community

answered Jan 12, 2019 at 5:01
Akash Kandpal 3,236 28 25

[@AssafJvone](#) why not? Would you mind to expand a bit on that? – monkey intern Nov 6, 2019 at 7:24

```
from collections import defaultdict

class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self,u,v):
        self.graph[u].append(v)

    def DFS(self,v,visited):
        visited.add(v)
        print(self.graph[v], "====")
        self.DFSUtil(v,visited)

    def DFSUtil(self,v,visited):
        visited.add(v)
        print(v)

        for i in self.graph[v]:
            if i not in visited:
                self.DFSUtil(i,visited)

g = Graph()

vertex=7
g.addEdge(0,1)
g.addEdge(0,2)
g.addEdge(0,3)
g.addEdge(0,4)
g.addEdge(1,4)
g.addEdge(2,4)
g.addEdge(3,4)
g.addEdge(4,5)
g.addEdge(5,6)

g.DFS(0,visited)
```

This is the modification for the above code because that doesn't work with in all cases. We have to specify the number of vertices and then give edges manually.

Share Edit Follow Flag

edited Oct 25, 2018 at 11:55
Naka 4,690 4 23 42

answered Oct 25, 2018 at 11:48
Aditya Ch 26 2

[@](#) which cases does the above code fail for? – Gulzar Aug 4, 2022 at 7:38

```
graph = {'A': ['B', 'C'],
        'B': ['A', 'D', 'E'],
        'C': ['A', 'F'],
        'D': ['B'],
        'E': ['B', 'F'],
        'F': ['C', 'E']}

def dfs(v,d):
    def dfs_helper(v,d):
        if v == d:
            return True
        if v in visited:
            return False
        visited.add(v)
        for i in graph[v]:
            dfs_helper(i,d)
        return False
    visited = set()
    return dfs_helper(v,d)

dfs('A','E') ---- True
dfs('A','B') ---- False
```

Share Edit Follow Flag

answered Sep 11, 2018 at 19:28
Sneha Mule 637 9 6

I think you have an indentation problem there. Assuming your code looks like this:

```
graph1 = {
    'A': ['B', 'S'],
    'B': ['A'],
    'C': ['B', 'E', 'F', 'S'],
    'D': ['C'],
    'E': ['C', 'A'],
    'F': ['C', 'A'],
    'S': ['A', 'E', 'G']}

visited = []

def dfs(graph,node):
    global visited
    if node not in visited:
        visited.append(node)
        for i in graph[node]:
            dfs(graph,i)

dfs(graph1,'A')
print(visited)
```

I would say a couple of things:

- Avoid globals if you can
- Instead of a list for visited, use a set

plus:

- This will not work for forests but I assume you already know that
- It will fail if you reference a node that does not exist.

Updated code:

```
graph1 = {
    'A': ['B', 'S'],
    'B': ['A'],
    'C': ['B', 'E', 'F', 'S'],
    'D': ['C'],
    'E': ['C', 'A'],
    'F': ['C', 'A'],
    'S': ['A', 'E', 'G']}

def dfs(graph, node, visited):
    if node not in visited:
        visited.append(node)
        for i in graph[node]:
            dfs(graph,i, visited)
        return visited
    visited = dfs(graph1,'A', [])
    print(visited)
```

Share Edit Follow Flag

edited Jul 26, 2017 at 15:31

answered May 9, 2017 at 11:55
Juan Luis 7,288 6 58 90

[@](#) Should the updated code contain the variable visited as dict or list? – Sudarshan Jul 23, 2017 at 5:26

[@Sudarshan](#) since it says visited.append(node) the variable visited is a list. – oamandaw Mar 2, 2020 at 1:27

[@Juan Luis](#) or anyone listening in the for loop of your code, how is variable i not stuck in values between 'B' and 'A' and how come it jumps to 'S' in the third iteration? – MM Khan Apr 25, 2020 at 18:58

[@](#) visited should be a set to reduce runtime by o(n) – Gulzar Aug 4, 2022 at 7:34

[@](#) making visited a bit vector reduces checks to O(1), but one could instead store for every vertex the adjacent vertices in sets and remove visited vertices directly from them. – user261818 Jun 30, 2023 at 9:22

[This post is hidden. It was deleted 4 years ago by the post author.](#)

Non recursive way.

```
def dfs(graph, node):
    visited = [node]
    stack = [node]
    while stack:
        top = stack.pop()
        print(top)
        for adj in graph[top]:
            if adj not in visited:
                visited.append(adj)
                stack.append(adj)
```

```
graph1 = {
    'A': ['B', 'S'],
    'B': ['A'],
    'C': ['B', 'E', 'F', 'S'],
    'D': ['C'],
    'E': ['C', 'A'],
    'F': ['C', 'A'],
    'S': ['A', 'E', 'G']}

dfs(graph1, 'A')
```

Share Edit Follow Undelete Flag

answered Mar 8, 2020 at 21:56
Sukashini 1,241 2 24 48

Comments disabled on deleted / locked posts / reviews